

Chapter 7

Advanced SQL

Database Systems:
Design, Implementation, and
Management, Sixth Edition, Rob and
Coronel

UNION Query

- ❑ Combine two companies customer lists where they have names in common so that the final list has no duplicate names

- ❑ Column names and their attributes must be the same

SELECT <COLUMN NAMES> FROM CUSTOMER

UNION

SELECT <COLUMN NAMES> FROM CUSTOMER_2;

- ❑ Can include more than two tables where applicable

SELECT <COLUMN NAMES> FROM T1 UNION

SELECT <COLUMN NAMES> FROM T2 UNION

SELECT <COLUMN NAMES> FROM T3;

UNION Query Result

FIGURE 7.1 UNION QUERY RESULT

The screenshot displays a database application interface with three windows. The top-left window shows the 'CUSTOMER' table with 10 records. The bottom-left window shows the 'CUSTOMER_2' table with 7 records. The right window shows the result of a UNION query, which combines the data from both tables, resulting in 15 records. The UNION query result starts with the data from 'CUSTOMER_2' followed by the data from 'CUSTOMER'.

CUSTOMER : Table

	CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_BALANCE
+	10010	Ramas	Alfred	A	615	844-2573	\$0.00
+	10011	Dunne	Leona	K	713	894-1238	\$0.00
+	10012	Smith	Kathy	W	615	894-2285	\$345.86
+	10013	Olowski	Paul	F	615	894-2180	\$536.75
+	10014	Orlando	Myron		615	222-1672	\$0.00
+	10015	O'Brian	Amy	B	713	442-3381	\$0.00
+	10016	Brown	James	G	615	297-1228	\$221.19
+	10017	Williams	George		615	290-2556	\$768.93
+	10018	Farriss	Anne	G	713	382-7185	\$216.55
+	10019	Smith	Olette	K	615	297-3809	\$0.00

Record: 1 of 10

CUSTOMER_2 : Table

	CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE
	345	Terrell	Justine	H	615	322-9870
	347	Olowski	Paul	F	615	894-2180
	351	Hernandez	Carlos	J	723	123-7654
	352	McDowell	George		723	123-7768
	365	Tirpin	Khaleed	G	723	123-9876
	368	Lewis	Marie	J	734	332-1789
	369	Dunne	Leona	K	713	894-1238

Record: 1 of 7

qryUNION-of-CUSTOMER-and-CUSTOMER_2 : Union Query

	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE
	Brown	James	G	615	297-1228
	Dunne	Leona	K	713	894-1238
	Farriss	Anne	G	713	382-7185
	Hernandez	Carlos	J	723	123-7654
	Lewis	Marie	J	734	332-1789
	McDowell	George		723	123-7768
	O'Brian	Amy	B	713	442-3381
	Olowski	Paul	F	615	894-2180
	Orlando	Myron		615	222-1672
	Ramas	Alfred	A	615	844-2573
	Smith	Kathy	W	615	894-2285
	Smith	Olette	K	615	297-3809
	Terrell	Justine	H	615	322-9870
	Tirpin	Khaleed	G	723	123-9876
	Williams	George		615	290-2556

Record: 1 of 15

UNION ALL Query Result

FIGURE 7.2 UNION ALL QUERY RESULT

The screenshot displays three database windows. The top-left window shows the 'CUSTOMER' table with 10 records. The bottom-left window shows the 'CUSTOMER_2' table with 7 records. The right window shows the result of a 'UNION ALL' query, which combines all records from both tables, resulting in 17 records, including duplicates.

CUSTOMER : Table

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_BALANCE
10010	Ramas	Alfred	A	615	844-2573	\$0.00
10011	Dunne	Leona	K	713	894-1238	\$0.00
10012	Smith	Kathy	vW	615	894-2285	\$3.
10013	Olowski	Paul	F	615	894-2180	\$5.
10014	Orlando	Myron		615	222-1672	:
10015	O'Brian	Amy	B	713	442-3381	:
10016	Brown	James	G	615	297-1228	\$2.
10017	Williams	George		615	290-2556	\$71
10018	Farriss	Anne	G	713	382-7185	\$2.
10019	Smith	Olette	K	615	297-3809	:

Record: 1 of 10

CUSTOMER_2 : Table

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE
345	Terrell	Justine	H	615	322-9870
347	Olowski	Paul	F	615	894-2180
351	Hernandez	Carlos	J	723	123-7654
352	McDowell	George		723	123-7768
365	Tirpin	Khaleed	G	723	123-9876
368	Lewis	Marie	J	734	332-1789
369	Dunne	Leona	K	713	894-1238

Record: 1 of 7

qryUNION-ALL-for-CUSTOMER-and-CUSTOMER_2 : Union Query

CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE
Ramas	Alfred	A	615	844-2573
Dunne	Leona	K	713	894-1238
Smith	Kathy	vW	615	894-2285
Olowski	Paul	F	615	894-2180
Orlando	Myron		615	222-1672
O'Brian	Amy	B	713	442-3381
Brown	James	G	615	297-1228
Williams	George		615	290-2556
Farriss	Anne	G	713	382-7185
Smith	Olette	K	615	297-3809
Terrell	Justine	H	615	322-9870
Olowski	Paul	F	615	894-2180
Hernandez	Carlos	J	723	123-7654
McDowell	George		723	123-7768
Tirpin	Khaleed	G	723	123-9876
Lewis	Marie	J	734	332-1789
Dunne	Leona	K	713	894-1238

Record: 1 of 17

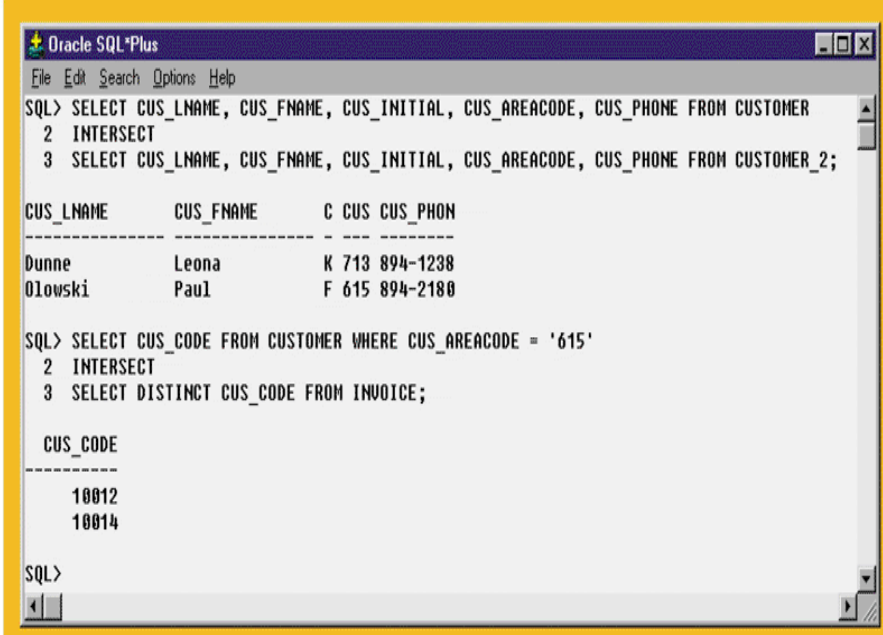
UNION ALL merges the tables but keeps duplicates in the result table

Intersect Query

- ❑ Find the rows that are in both tables
- ❑ Find rows based on another select
- ❑ Not available in MS Access
- ❑ Alternative

```
SELECT CUS_CODE FROM  
CUSTOMER  
WHERE CUS_AREACODE='615'  
AND CUS_CODE IN  
(SELECT DISTINCT  
CUS_CODE FROM INVOICE) ;
```

FIGURE 7.3 INTERSECT QUERY RESULT



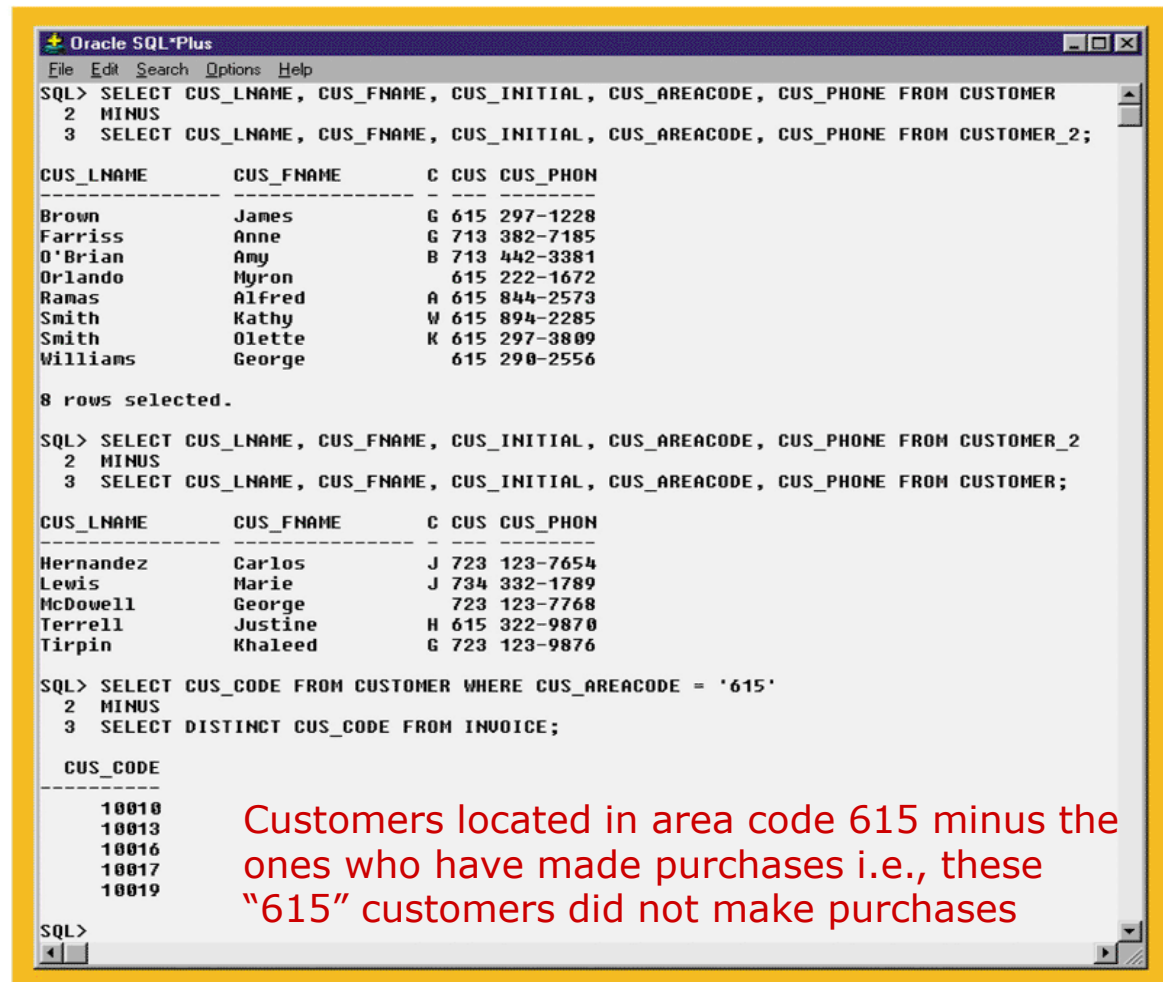
The screenshot shows the Oracle SQL*Plus interface with a menu bar (File, Edit, Search, Options, Help) and a command window. The first query is an INTERSECT query that selects customer details from two tables. The result is displayed in a table with columns CUS_LNAME, CUS_FNAME, C, CUS_CODE, and CUS_PHONE. The second query is another INTERSECT query that selects distinct customer codes from the CUSTOMER and INVOICE tables. The result shows two customer codes: 10012 and 10014.

```
Oracle SQL*Plus  
File Edit Search Options Help  
SQL> SELECT CUS_LNAME, CUS_FNAME, CUS_INITIAL, CUS_AREACODE, CUS_PHONE FROM CUSTOMER  
2 INTERSECT  
3 SELECT CUS_LNAME, CUS_FNAME, CUS_INITIAL, CUS_AREACODE, CUS_PHONE FROM CUSTOMER_2;  
  
CUS_LNAME      CUS_FNAME      C CUS_CODE CUS_PHONE  
-----  
Dunne          Leona          K 713 894-1238  
Olowski        Paul           F 615 894-2180  
  
SQL> SELECT CUS_CODE FROM CUSTOMER WHERE CUS_AREACODE = '615'  
2 INTERSECT  
3 SELECT DISTINCT CUS_CODE FROM INVOICE;  
  
CUS_CODE  
-----  
10012  
10014  
  
SQL>
```

Minus Query

FIGURE 7.4 MINUS QUERY RESULTS

- Combines rows from two queries and returns only those rows that appear in the first set but not the second
- Alternative – use **NOT IN**



```
Oracle SQL*Plus
File Edit Search Options Help
SQL> SELECT CUS_LNAME, CUS_FNAME, CUS_INITIAL, CUS_AREACODE, CUS_PHONE FROM CUSTOMER
2 MINUS
3 SELECT CUS_LNAME, CUS_FNAME, CUS_INITIAL, CUS_AREACODE, CUS_PHONE FROM CUSTOMER_2;

CUS_LNAME      CUS_FNAME      C CUS CUS_PHON
-----
Brown          James          G 615 297-1228
Farriss        Anne          G 713 382-7185
O'Brian        Amy           B 713 442-3381
Orlando        Myron         615 222-1672
Ramas          Alfred        A 615 844-2573
Smith          Kathy         W 615 894-2285
Smith          Olette        K 615 297-3809
Williams       George        615 298-2556

8 rows selected.

SQL> SELECT CUS_LNAME, CUS_FNAME, CUS_INITIAL, CUS_AREACODE, CUS_PHONE FROM CUSTOMER_2
2 MINUS
3 SELECT CUS_LNAME, CUS_FNAME, CUS_INITIAL, CUS_AREACODE, CUS_PHONE FROM CUSTOMER;

CUS_LNAME      CUS_FNAME      C CUS CUS_PHON
-----
Hernandez      Carlos          J 723 123-7654
Lewis          Marie          J 734 332-1789
McDowell       George         723 123-7768
Terrell        Justine        H 615 322-9870
Tirpin         Khaleed        G 723 123-9876

SQL> SELECT CUS_CODE FROM CUSTOMER WHERE CUS_AREACODE = '615'
2 MINUS
3 SELECT DISTINCT CUS_CODE FROM INVOICE;

CUS_CODE
-----
10010
10013
10016
10017
10019
```

Customers located in area code 615 minus the ones who have made purchases i.e., these "615" customers did not make purchases

SQL Join Expression Styles

TABLE 7.1 SQL JOIN EXPRESSION STYLES

JOIN CLASSIFICATION	JOIN TYPE	SQL SYNTAX EXAMPLE	DESCRIPTION
Cross	CROSS JOIN	SELECT * FROM T1, T2	Returns the Cartesian product of T1 and T2—old style
		SELECT * FROM T1 CROSS JOIN T2	Returns the Cartesian product of T1 and T2
Inner	Old-Style JOIN	SELECT * FROM T1, T2 WHERE T1.C1=T2.C1	Returns only the rows that meet the join condition in the WHERE clause—old style. Only rows with matching values are selected.
	NATURAL JOIN	SELECT * FROM T1 NATURAL JOIN T2	Returns only the rows with matching values in the matching columns. The matching columns must have the same names and similar data types.
	JOIN USING	SELECT * FROM T1 JOIN T2 USING (C1)	Returns only the rows with matching values in the columns indicated in the USING clause
	JOIN ON	SELECT * FROM T1 JOIN T2 ON T1.C1=T2.C1	Returns only the rows that meet the join condition indicated in the ON clause
Outer	LEFT JOIN	SELECT * FROM T1 LEFT OUTER JOIN T2 ON T1.C1=T2.C1	Returns rows with matching values and includes all rows from the left table (T1) with unmatched values
	RIGHT JOIN	SELECT * FROM T1 RIGHT OUTER JOIN T2 ON T1.C1=T2.C1	Returns rows with matching values and includes all rows from the right table (T2) with unmatched values
	FULL JOIN	SELECT * FROM T1 FULL OUTER JOIN T2 ON T1.C1=T2.C1	Returns rows with matching values and includes all rows from both tables (T1 and T2) with unmatched values

Additional clauses with JOIN

- ❑ JOIN USING – return only the rows with matching values in the column indicated in the USING clause
 - This column must exist in both tables
 - SELECT column-list FROM table1 JOIN table2 USING (common-column)
- ❑ JOIN ON – when the tables have no common attribute name but the columns have the same attribute type
 - SELECT column-list FROM table1 JOIN table2 ON join-condition

SELECT Subquery Examples

TABLE 7.2 SELECT SUBQUERY EXAMPLES

SELECT SUBQUERY EXAMPLES	EXPLANATION
<pre>INSERT INTO PRODUCT SELECT * FROM P;</pre>	Inserts all rows from the table P into the PRODUCT Table. Both tables must have the same attributes. The subquery returns all rows from table P.
<pre>UPDATE PRODUCT SET P_PRICE = (SELECT AVG(P_PRICE) FROM PRODUCT) WHERE V_CODE IN (SELECT V_CODE FROM VENDOR WHERE V_AREACODE = '615');</pre>	Updates the product price to the average product price, but only for the products that are provided by vendors who have an area code equal to 615. The first subquery returns the average price; the second subquery returns the list of vendors with an area code equal to 615.
<pre>DELETE FROM PRODUCT WHERE V_CODE IN (SELECT V_CODE FROM VENDOR WHERE V_AREACODE = '615');</pre>	Deletes the PRODUCT table rows that are provided by vendors with an area code equal to '615'. The subquery returns the list of vendors' codes with area code equal to 615.

WHERE Subquery Examples

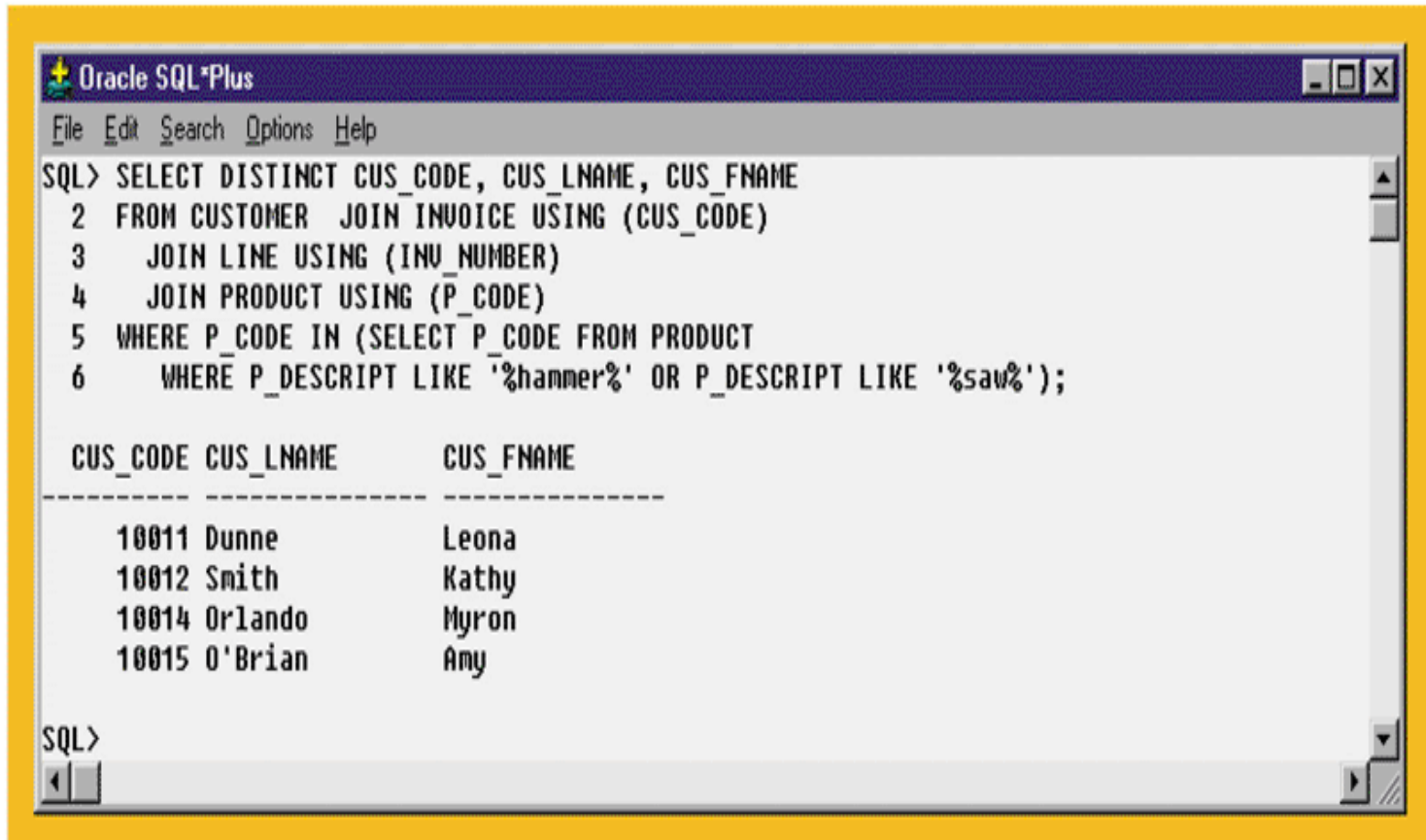
- ❑ Subquery can return
 - One single value (one column, one row)
 - A list of values (one column, multiple rows)
 - A virtual table (multicolumn, multirow set of values)
 - NULL

Other types of subqueries

- IN subqueries – you want to find all customers who have purchased something from a list of products, not just one product
- HAVING subqueries – restrict a GROUP BY by applying a conditional criteria to the grouped rows
 - List all products with the total quantity sold greater than the average quantity sold
- INLINE subqueries – can appear as a value in a the columnlist of a SELECT
 - Must return one value
 - Compute difference between product's individual price and its average price
- CORRELATED subqueries – subquery executes once for each row in the outer row (like nested loops)

IN Subquery Example

FIGURE 7.14 IN SUBQUERY EXAMPLE



The screenshot shows the Oracle SQL*Plus interface. The title bar reads "Oracle SQL*Plus". The menu bar includes "File", "Edit", "Search", "Options", and "Help". The command window contains the following SQL query:

```
SQL> SELECT DISTINCT CUS_CODE, CUS_LNAME, CUS_FNAME
2  FROM CUSTOMER JOIN INVOICE USING (CUS_CODE)
3  JOIN LINE USING (INV_NUMBER)
4  JOIN PRODUCT USING (P_CODE)
5  WHERE P_CODE IN (SELECT P_CODE FROM PRODUCT
6  WHERE P_DESCRIPT LIKE '%hammer%' OR P_DESCRIPT LIKE '%saw%');
```

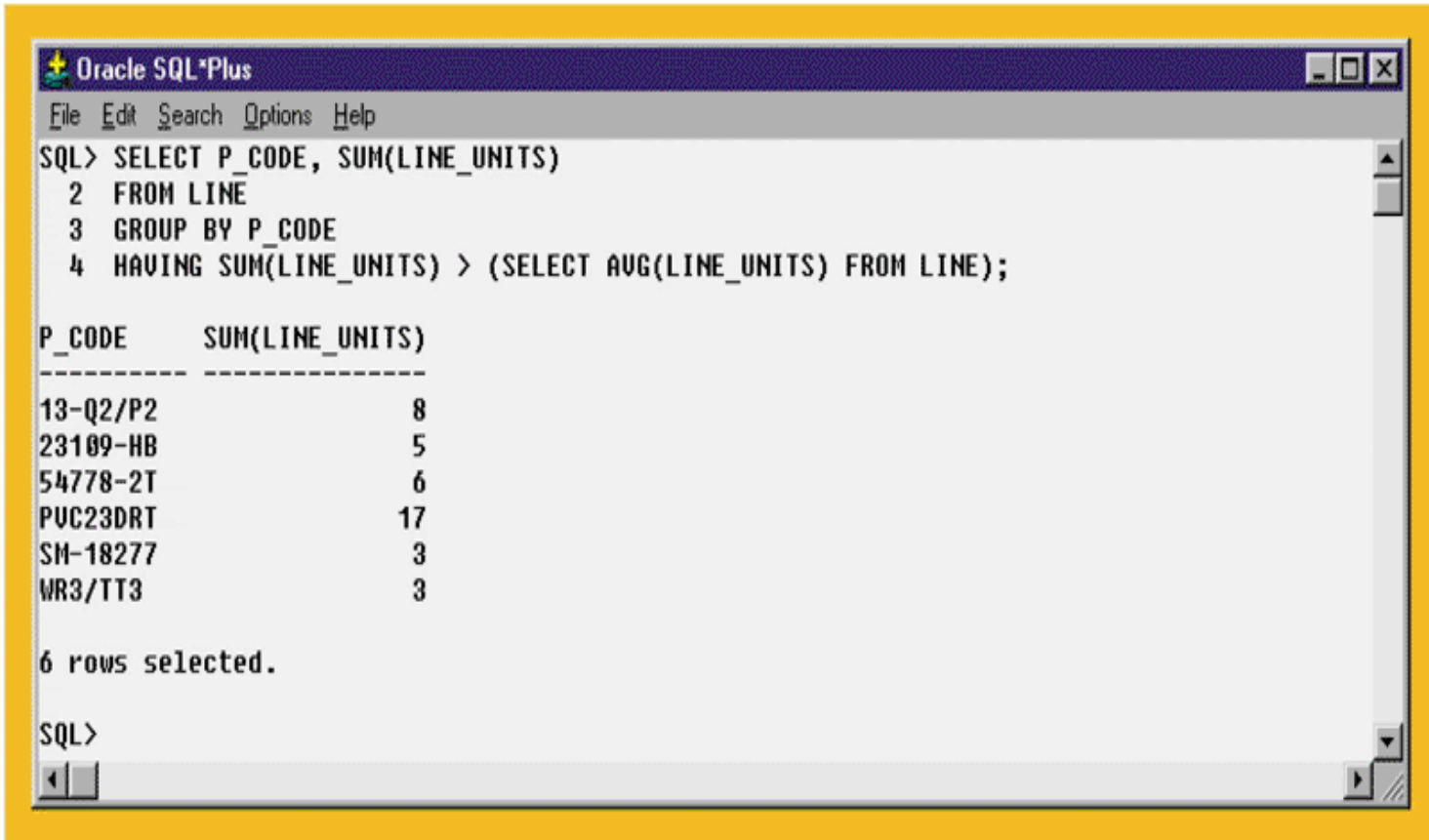
The query results are displayed in a table with three columns: CUS_CODE, CUS_LNAME, and CUS_FNAME. The results are as follows:

CUS_CODE	CUS_LNAME	CUS_FNAME
10011	Dunne	Leona
10012	Smith	Kathy
10014	Orlando	Myron
10015	O'Brian	Amy

The command window ends with "SQL>" and a cursor.

HAVING Subquery Example

FIGURE 7.15 HAVING SUBQUERY EXAMPLE



The screenshot shows the Oracle SQL*Plus interface. The title bar reads "Oracle SQL*Plus". The menu bar includes "File", "Edit", "Search", "Options", and "Help". The command window contains the following SQL query:

```
SQL> SELECT P_CODE, SUM(LINE_UNITS)
2  FROM LINE
3  GROUP BY P_CODE
4  HAVING SUM(LINE_UNITS) > (SELECT AVG(LINE_UNITS) FROM LINE);
```

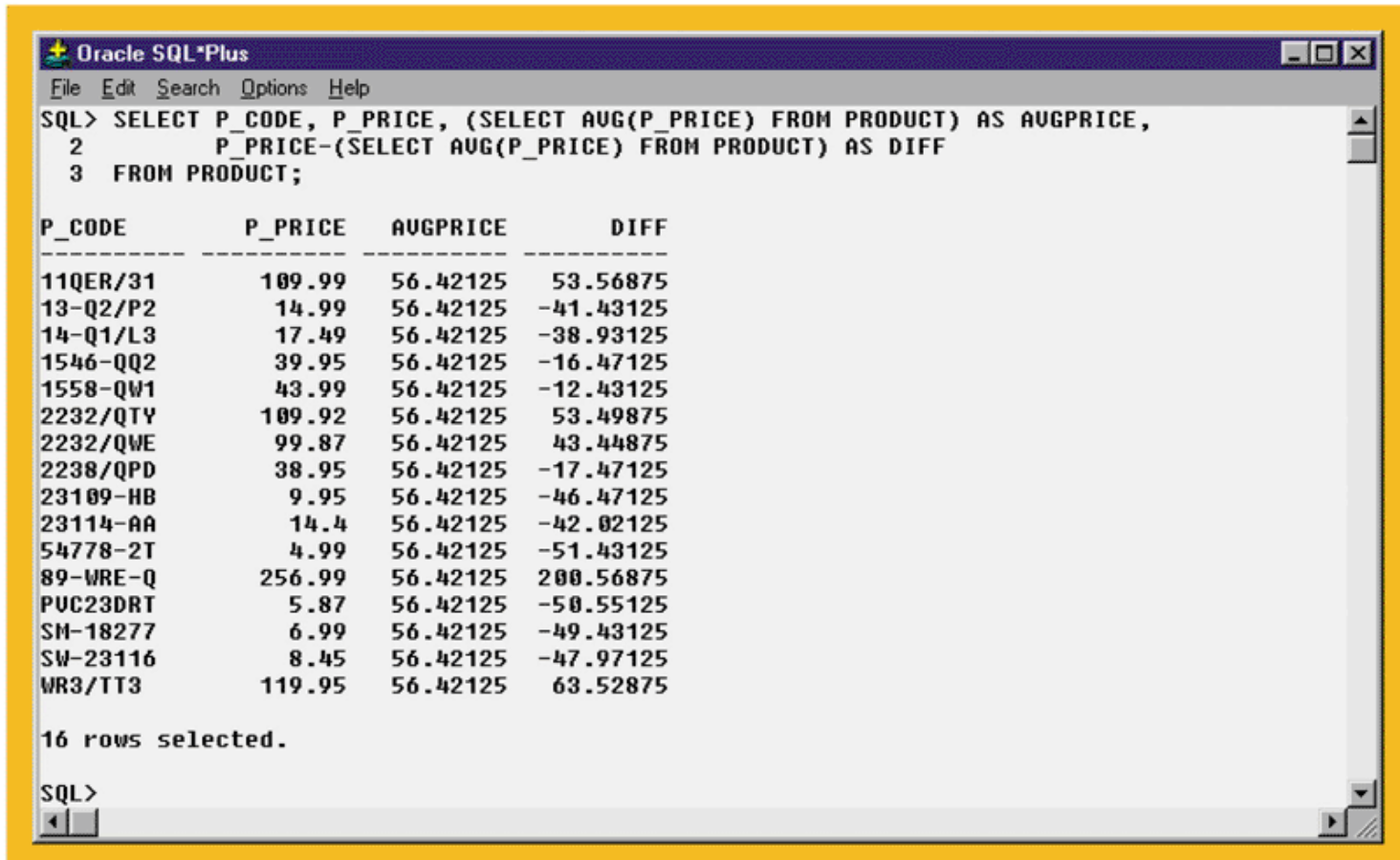
The results are displayed in a table with two columns: "P_CODE" and "SUM(LINE_UNITS)". The data is as follows:

P_CODE	SUM(LINE_UNITS)
13-Q2/P2	8
23109-HB	5
54778-2T	6
PVC23DRT	17
SM-18277	3
WR3/TT3	3

Below the table, it says "6 rows selected." The prompt "SQL>" is visible at the bottom of the command window.

Inline Subquery Example

FIGURE 7.18 INLINE SUBQUERY EXAMPLE



The screenshot shows the Oracle SQL*Plus interface. The command window contains the following SQL query:

```
SQL> SELECT P_CODE, P_PRICE, (SELECT AVG(P_PRICE) FROM PRODUCT) AS AVGPRICE,  
2      P_PRICE-(SELECT AVG(P_PRICE) FROM PRODUCT) AS DIFF  
3 FROM PRODUCT;
```

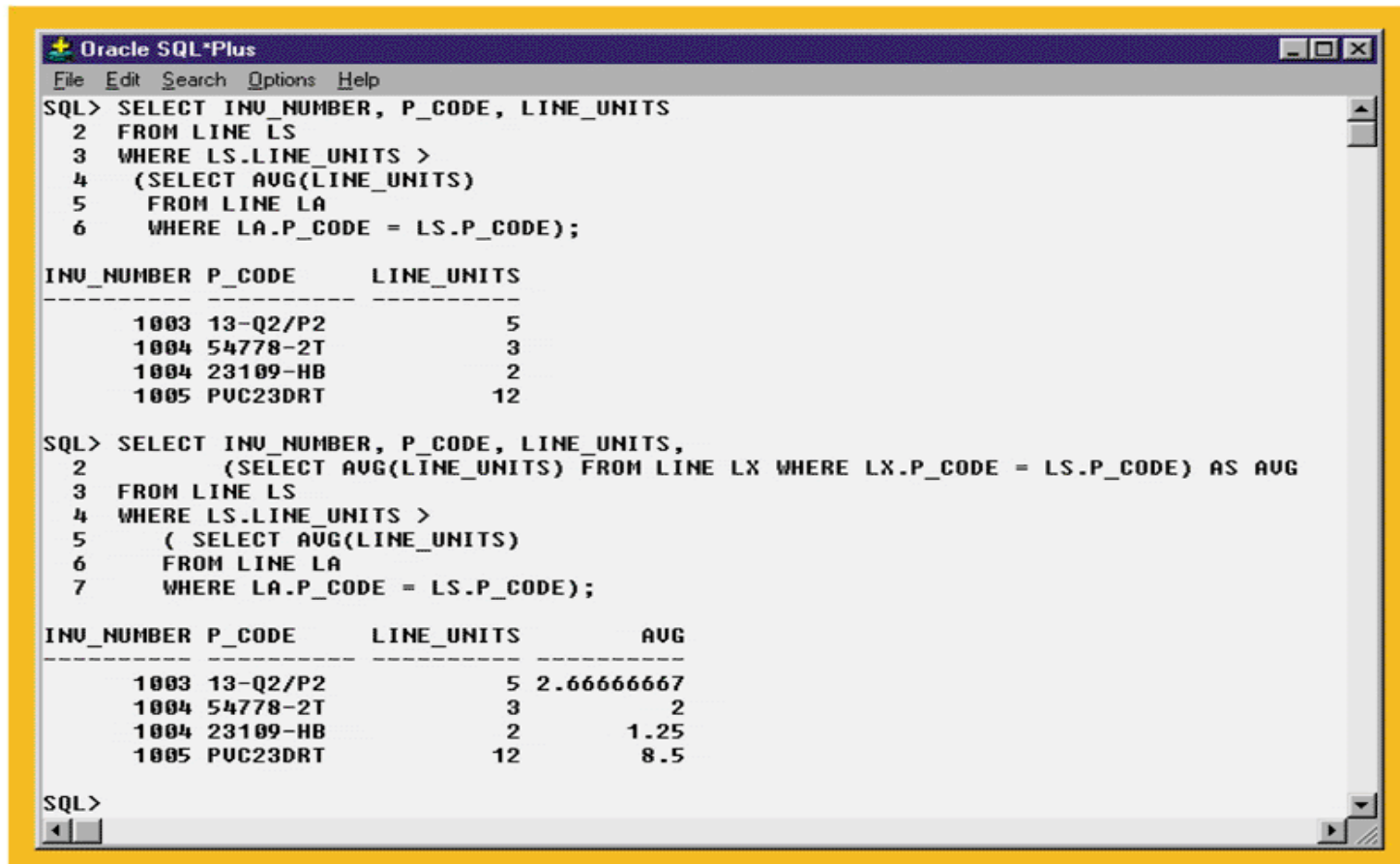
The result set displays 16 rows of data with the following columns: P_CODE, P_PRICE, AVGPRICE, and DIFF. The AVGPRICE column shows a constant value of 56.42125 for all rows, representing the average price of all products. The DIFF column shows the difference between each product's price and this average.

P_CODE	P_PRICE	AVGPRICE	DIFF
11QER/31	109.99	56.42125	53.56875
13-Q2/P2	14.99	56.42125	-41.43125
14-Q1/L3	17.49	56.42125	-38.93125
1546-QQ2	39.95	56.42125	-16.47125
1558-QW1	43.99	56.42125	-12.43125
2232/QTU	109.92	56.42125	53.49875
2232/QWE	99.87	56.42125	43.44875
2238/QPD	38.95	56.42125	-17.47125
23109-HB	9.95	56.42125	-46.47125
23114-AA	14.4	56.42125	-42.02125
54778-2T	4.99	56.42125	-51.43125
89-WRE-Q	256.99	56.42125	200.56875
PUC23DRT	5.87	56.42125	-50.55125
SM-18277	6.99	56.42125	-49.43125
SW-23116	8.45	56.42125	-47.97125
WR3/TT3	119.95	56.42125	63.52875

16 rows selected.

Correlated Subquery Examples

FIGURE 7.20 CORRELATED SUBQUERY EXAMPLES



The screenshot shows the Oracle SQL*Plus interface with two SQL queries and their results. The first query filters for line units greater than the average of line units for the same product code. The second query adds a column showing the average line units for each product code.

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> SELECT INU_NUMBER, P_CODE, LINE_UNITS
2 FROM LINE LS
3 WHERE LS.LINE_UNITS >
4 (SELECT AVG(LINE_UNITS)
5 FROM LINE LA
6 WHERE LA.P_CODE = LS.P_CODE);
```

INU_NUMBER	P_CODE	LINE_UNITS
1003	13-Q2/P2	5
1004	54778-2T	3
1004	23109-HB	2
1005	PVC23DRT	12

```
SQL> SELECT INU_NUMBER, P_CODE, LINE_UNITS,
2 (SELECT AVG(LINE_UNITS) FROM LINE LX WHERE LX.P_CODE = LS.P_CODE) AS AVG
3 FROM LINE LS
4 WHERE LS.LINE_UNITS >
5 ( SELECT AVG(LINE_UNITS)
6 FROM LINE LA
7 WHERE LA.P_CODE = LS.P_CODE);
```

INU_NUMBER	P_CODE	LINE_UNITS	AVG
1003	13-Q2/P2	5	2.66666667
1004	54778-2T	3	2
1004	23109-HB	2	1.25
1005	PVC23DRT	12	8.5

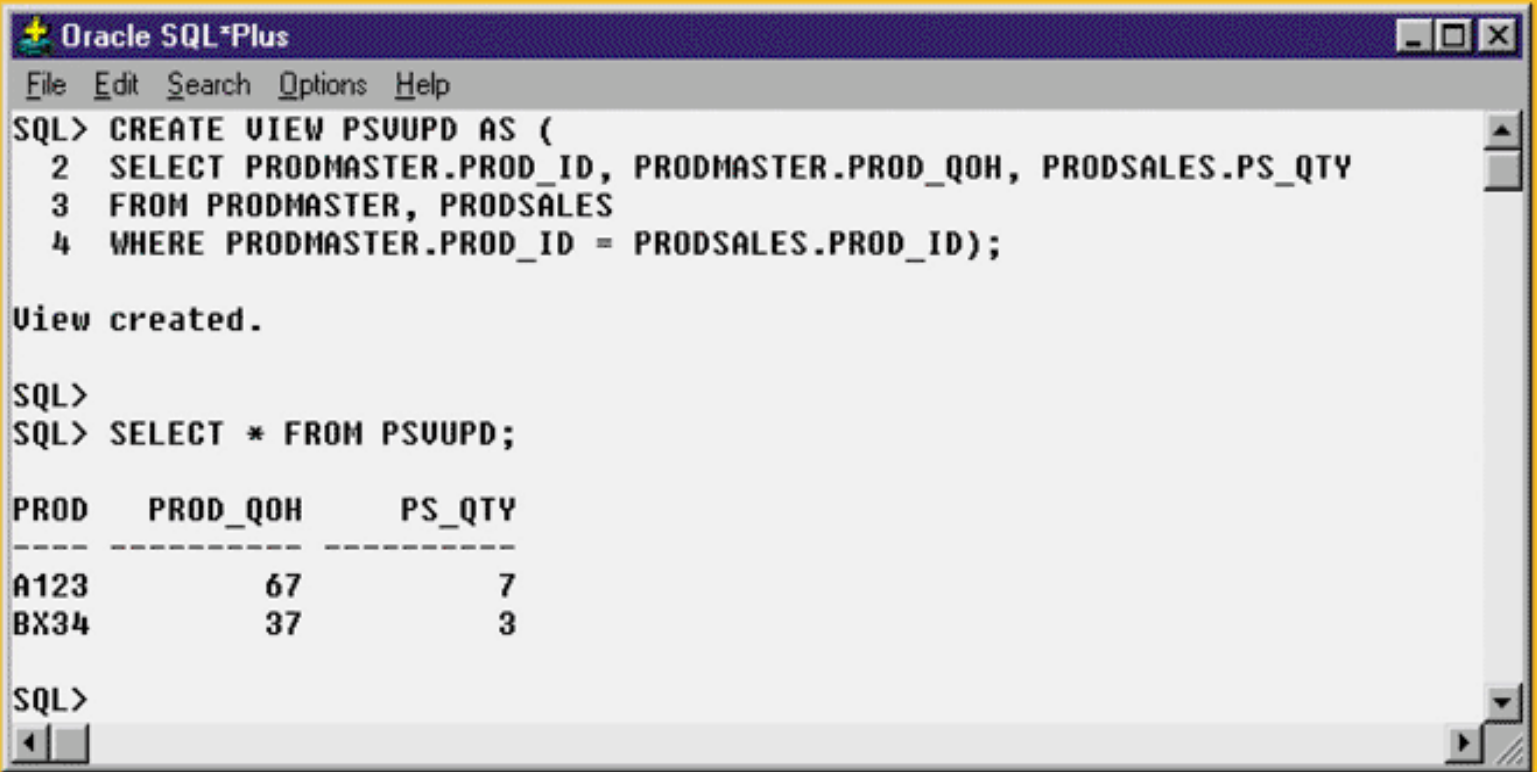
```
SQL>
```

Updatable Views

- ❑ Views can be made to serve common data management tasks
 - Batch update routine – pools multiple transactions into a single batch to update a master table file in a single operation
 - Updates attributes in the base tables that are used in the view
 - ❑ Update a product's available inventory based on summary sales transactions

Creating an Updatable View in Oracle

FIGURE 7.26 CREATING AN UPDATABLE VIEW IN ORACLE



The screenshot shows the Oracle SQL*Plus command-line interface. The user has entered a SQL command to create a view named PSUUPD. The command is: `CREATE VIEW PSUUPD AS (SELECT PRODMASTER.PROD_ID, PRODMASTER.PROD_QOH, PRODSALES.PS_QTY FROM PRODMASTER, PRODSALES WHERE PRODMASTER.PROD_ID = PRODSALES.PROD_ID);`. The response is "View created.". The user then enters a query: `SELECT * FROM PSUUPD;`. The result is a table with three columns: PROD, PROD_QOH, and PS_QTY. The data rows are A123 with PROD_QOH 67 and PS_QTY 7, and BX34 with PROD_QOH 37 and PS_QTY 3.

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> CREATE VIEW PSUUPD AS (
  2  SELECT PRODMASTER.PROD_ID, PRODMASTER.PROD_QOH, PRODSALES.PS_QTY
  3  FROM PRODMASTER, PRODSALES
  4  WHERE PRODMASTER.PROD_ID = PRODSALES.PROD_ID);

View created.

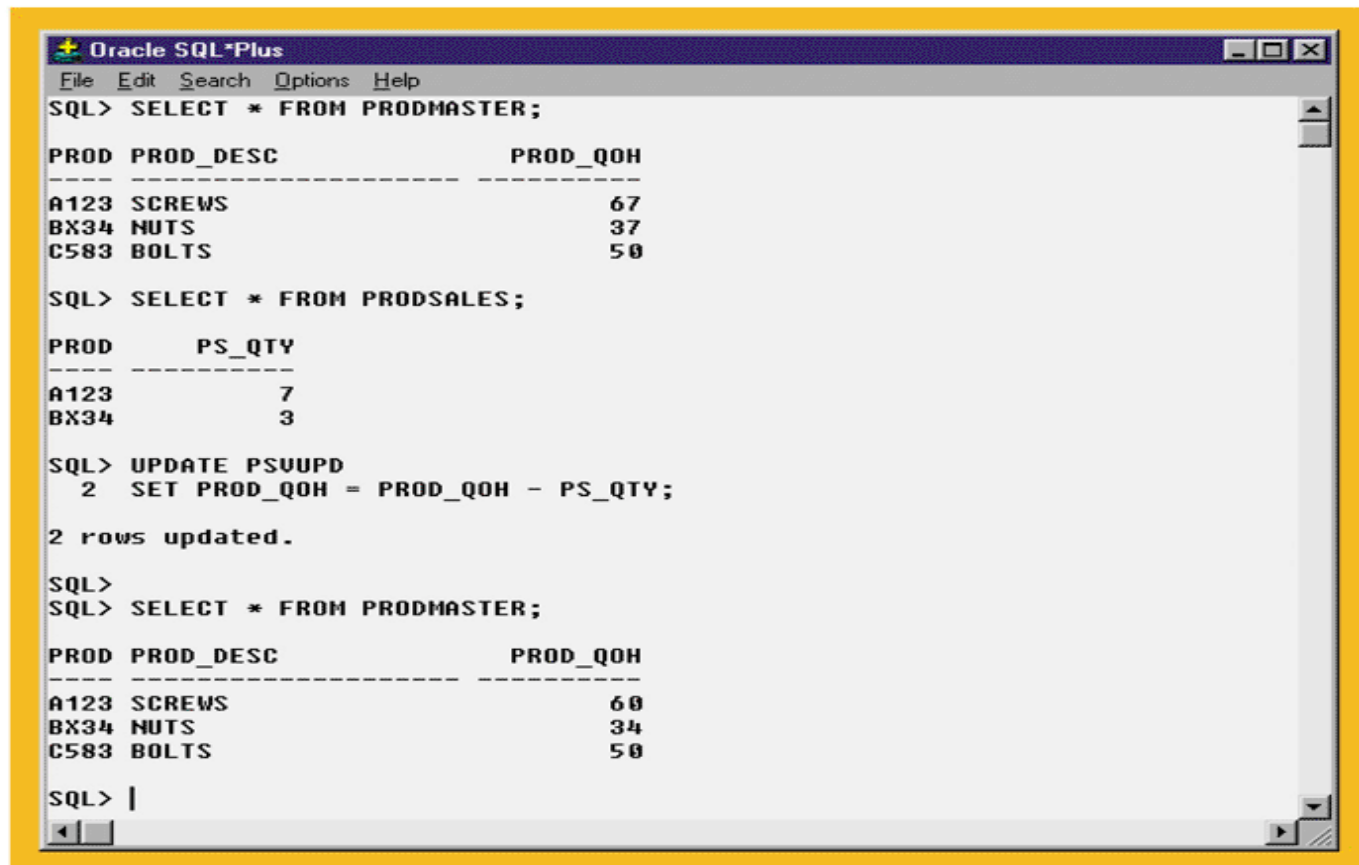
SQL>
SQL> SELECT * FROM PSUUPD;

PROD   PROD_QOH   PS_QTY
-----
A123         67         7
BX34         37         3

SQL>
```

PRODMASTER Table Update, Using an Updatable View

FIGURE 7.27 PRODMASTER TABLE UPDATE, USING AN UPDATABLE VIEW

The screenshot shows an Oracle SQL*Plus window with a menu bar (File, Edit, Search, Options, Help) and a command area. The user has entered several SQL commands. First, they selected all data from the PRODMASTER table, which returned three rows: A123 SCREWS with a quantity of 67, BX34 NUTS with 37, and C583 BOLTS with 50. Next, they selected all data from the PRODSALES table, which returned two rows: A123 with a quantity of 7 and BX34 with 3. Then, they executed an update statement: 'UPDATE PSUUPD 2 SET PROD_QOH = PROD_QOH - PS_QTY;'. The system responded with '2 rows updated.'. Finally, they selected all data from the PRODMASTER table again, and the results show that the quantities for A123 and BX34 have been updated to 60 and 34 respectively, while C583 remains at 50.

```
Oracle SQL*Plus
File Edit Search Options Help
SQL> SELECT * FROM PRODMASTER;

PROD PROD_DESC          PROD_QOH
-----
A123  SCREWS                  67
BX34  NUTS                    37
C583  BOLTS                    50

SQL> SELECT * FROM PRODSALES;

PROD    PS_QTY
-----
A123         7
BX34         3

SQL> UPDATE PSUUPD
      2 SET PROD_QOH = PROD_QOH - PS_QTY;

2 rows updated.

SQL>
SQL> SELECT * FROM PRODMASTER;

PROD PROD_DESC          PROD_QOH
-----
A123  SCREWS                  60
BX34  NUTS                    34
C583  BOLTS                    50

SQL> |
```

Procedural SQL

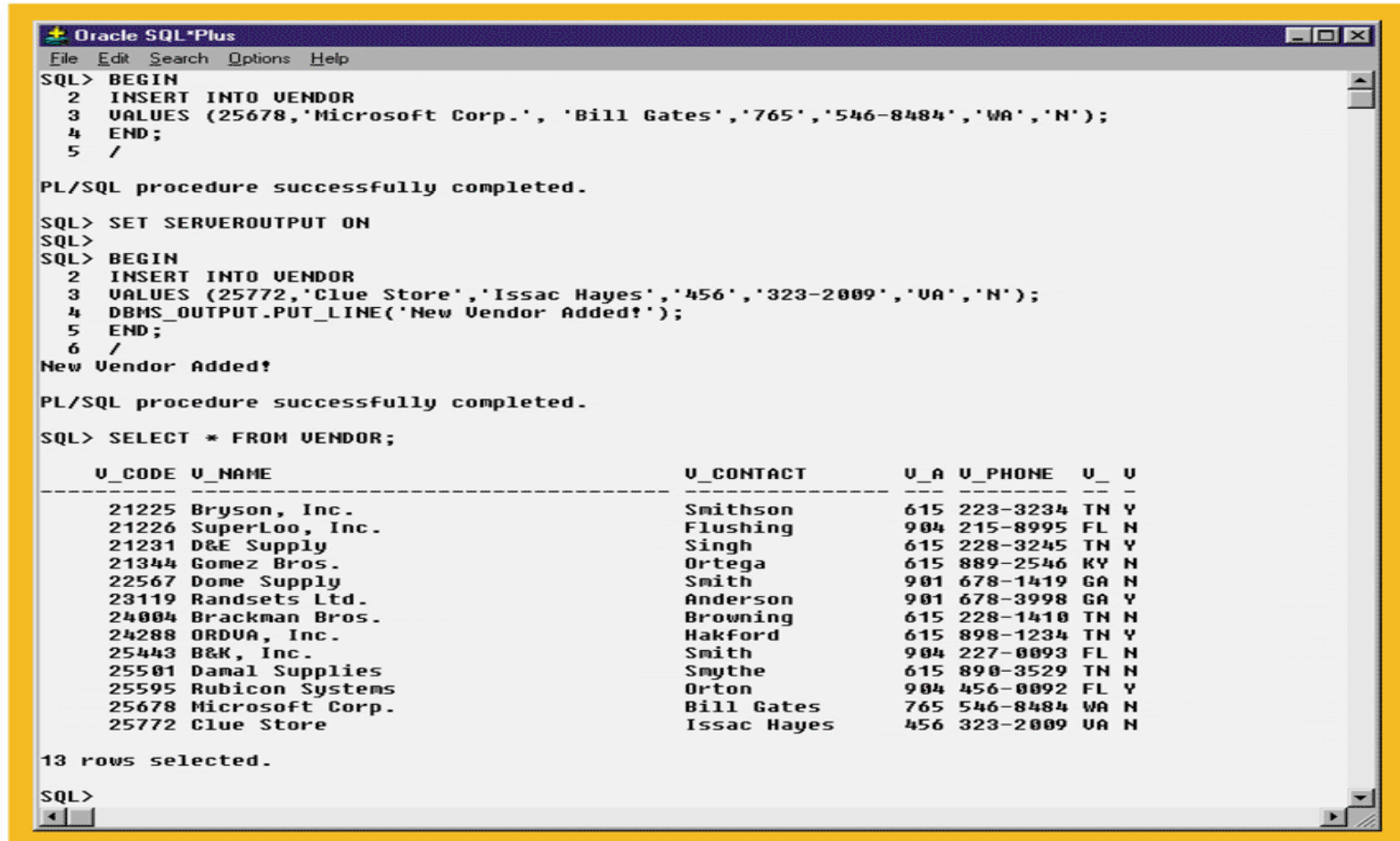
- ❑ SQL does not support the conditional execution of procedures (IF-THEN-ELSE) or loops
- ❑ One way to do this is by writing procedural code in a programming language and including SQL statements in the program
 - This spreads out SQL code in many programs and make changes difficult
- ❑ A better way is to isolate critical code and have all applications call this shared code
 - Needed by distributed and OO databases
 - SQL 99 defined the use of **persistent stored modules** (PSMs)

Procedural SQL

- ❑ A PSM is a block of code that is stored and executed at the DBMS server
 - Represents business logic that can be shared among multiple database users
 - Access can be controlled by the DBA
 - Oracle supports this through Procedural SQL – PL/SQL

Anonymous PL/SQL Block Examples

FIGURE 7.28 ANONYMOUS PL/SQL BLOCK EXAMPLES



```
Oracle SQL*Plus
File Edit Search Options Help
SQL> BEGIN
2  INSERT INTO VENDOR
3  VALUES (25678,'Microsoft Corp.', 'Bill Gates','765','546-8484','WA','N');
4  END;
5  /

PL/SQL procedure successfully completed.

SQL> SET SERVEROUTPUT ON
SQL>
SQL> BEGIN
2  INSERT INTO VENDOR
3  VALUES (25772,'Clue Store','Issac Hayes','456','323-2009','UA','N');
4  DBMS_OUTPUT.PUT_LINE('New Vendor Added!');
5  END;
6  /
New Vendor Added!

PL/SQL procedure successfully completed.

SQL> SELECT * FROM VENDOR;

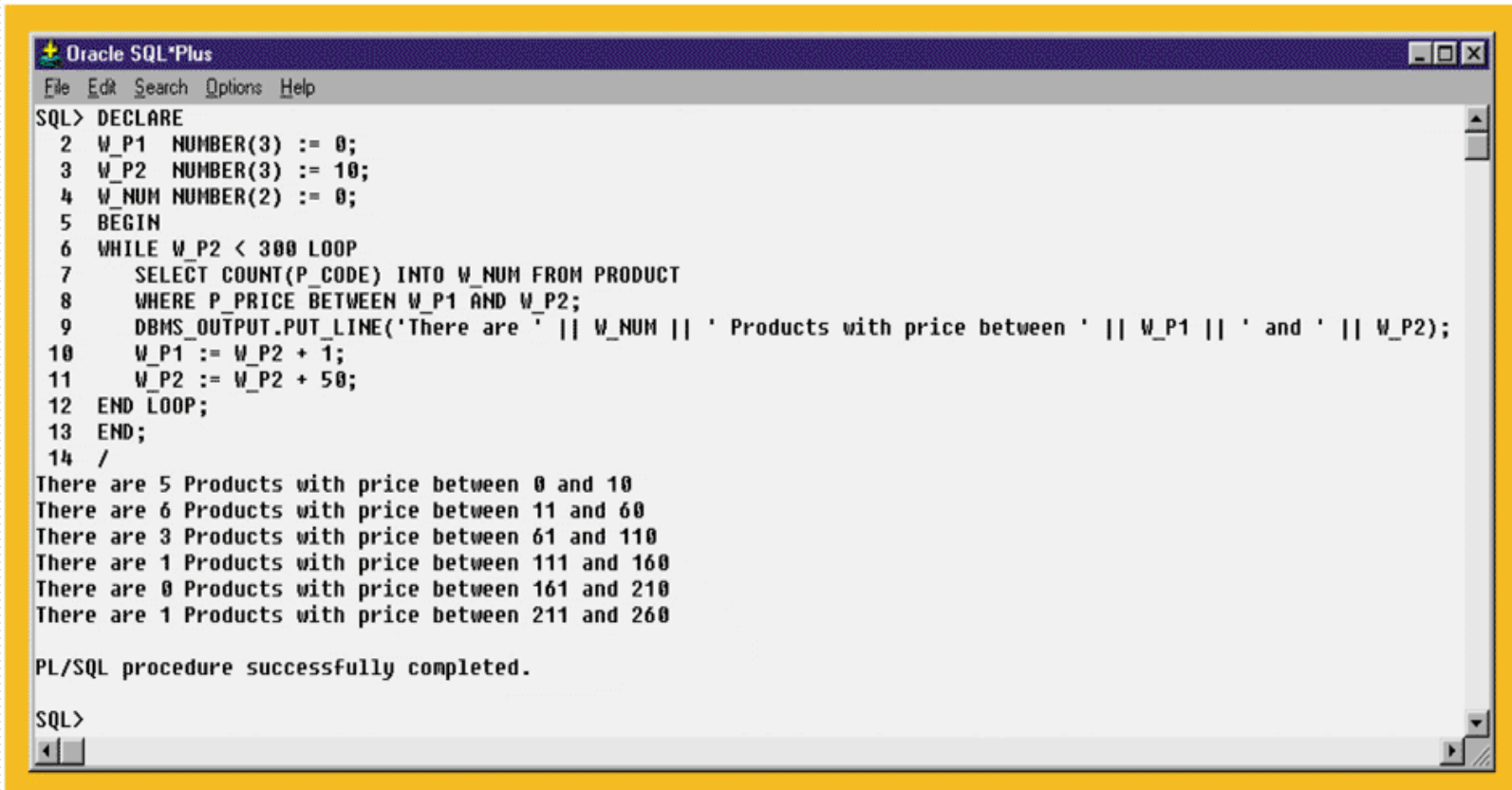
  U_CODE U_NAME                U_CONTACT      U_A U_PHONE  U_  U
-----
21225 Bryson, Inc.             Smithson        615 223-3234 TN Y
21226 SuperLoo, Inc.          Flushing       904 215-8995 FL N
21231 D&E Supply              Singh           615 228-3245 TN Y
21344 Gomez Bros.            Ortega          615 889-2546 KY N
22567 Dome Supply            Smith           901 678-1419 GA N
23119 Randsets Ltd.          Anderson        901 678-3998 GA Y
24004 Brackman Bros.          Browning        615 228-1410 TN N
24288 ORDUA, Inc.             Hakford         615 898-1234 TN Y
25443 B&K, Inc.               Smith           904 227-0093 FL N
25501 Damal Supplies          Smythe          615 890-3529 TN N
25595 Rubicon Systems         Orton           904 456-0092 FL Y
25678 Microsoft Corp.        Bill Gates      765 546-8484 WA N
25772 Clue Store              Issac Hayes     456 323-2009 UA N

13 rows selected.

SQL>
```

Anonymous PL/SQL Block with Variables and Loops

FIGURE 7.29 ANONYMOUS PL/SQL BLOCK WITH VARIABLES AND LOOPS

The image is a screenshot of the Oracle SQL*Plus command-line interface. The window has a title bar 'Oracle SQL*Plus' and a menu bar with 'File', 'Edit', 'Search', 'Options', and 'Help'. The main text area shows the execution of an anonymous PL/SQL block. The code starts with 'SQL> DECLARE' followed by variable declarations: 'W_P1 NUMBER(3) := 0;', 'W_P2 NUMBER(3) := 10;', and 'W_NUM NUMBER(2) := 0;'. It then begins with 'BEGIN' and enters a 'WHILE W_P2 < 300 LOOP'. Inside the loop, it executes a 'SELECT COUNT(P_CODE) INTO W_NUM FROM PRODUCT WHERE P_PRICE BETWEEN W_P1 AND W_P2;', followed by 'DBMS_OUTPUT.PUT_LINE('There are ' || W_NUM || ' Products with price between ' || W_P1 || ' and ' || W_P2);'. The loop increments 'W_P1 := W_P2 + 1;' and 'W_P2 := W_P2 + 50;'. The loop ends with 'END LOOP;' and the block ends with 'END;' and a slash '/'. The output shows six lines of results, each corresponding to a price range. The final message is 'PL/SQL procedure successfully completed.' and the prompt 'SQL>' is visible at the bottom.

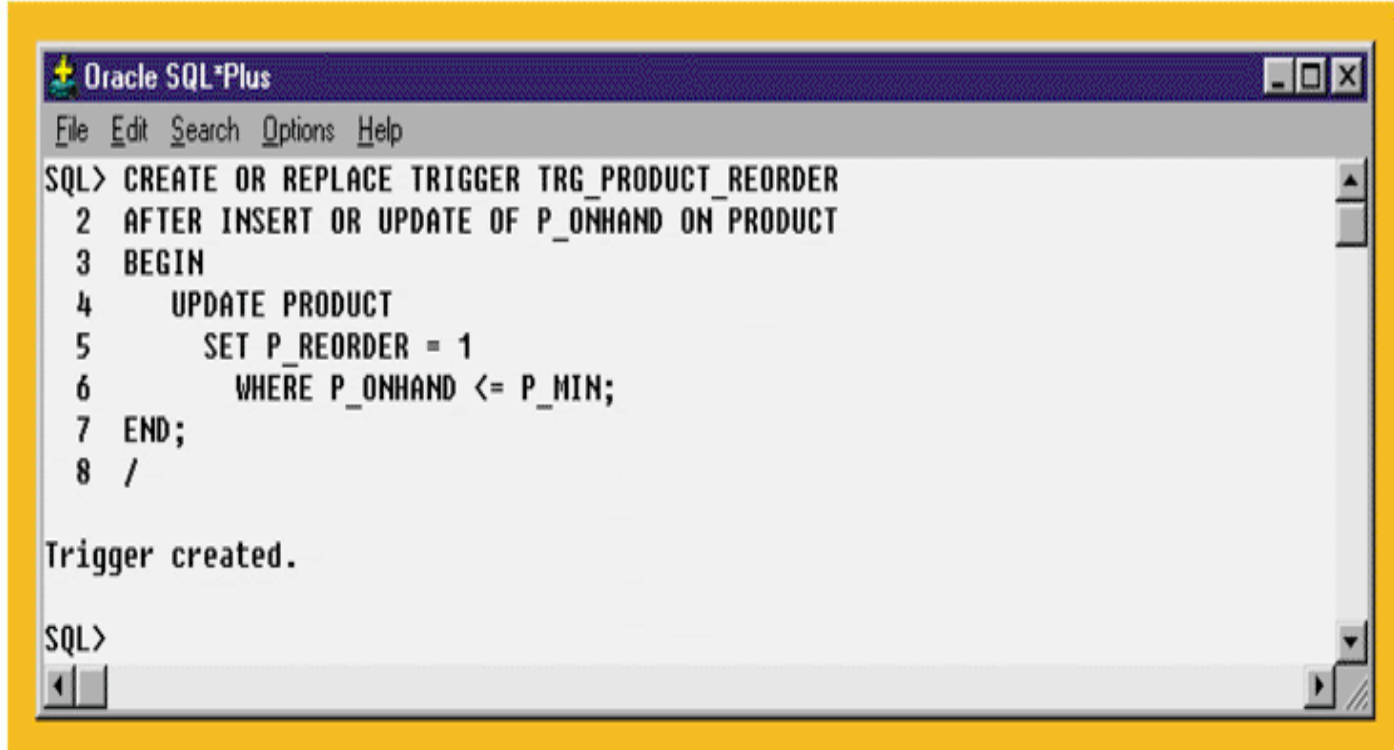
```
SQL> DECLARE
 2  W_P1  NUMBER(3) := 0;
 3  W_P2  NUMBER(3) := 10;
 4  W_NUM NUMBER(2) := 0;
 5  BEGIN
 6  WHILE W_P2 < 300 LOOP
 7      SELECT COUNT(P_CODE) INTO W_NUM FROM PRODUCT
 8      WHERE P_PRICE BETWEEN W_P1 AND W_P2;
 9      DBMS_OUTPUT.PUT_LINE('There are ' || W_NUM || ' Products with price between ' || W_P1 || ' and ' || W_P2);
10      W_P1 := W_P2 + 1;
11      W_P2 := W_P2 + 50;
12  END LOOP;
13  END;
14  /
There are 5 Products with price between 0 and 10
There are 6 Products with price between 11 and 60
There are 3 Products with price between 61 and 110
There are 1 Products with price between 111 and 160
There are 0 Products with price between 161 and 210
There are 1 Products with price between 211 and 260

PL/SQL procedure successfully completed.

SQL>
```

Creating a Trigger using PL/SQL

FIGURE 7.31 CREATING THE TRG_PRODUCT_REORDER TRIGGER

A screenshot of the Oracle SQL*Plus command-line interface. The window has a title bar that says "Oracle SQL*Plus" and a menu bar with "File", "Edit", "Search", "Options", and "Help". The main text area shows the following SQL commands:

```
SQL> CREATE OR REPLACE TRIGGER TRG_PRODUCT_REORDER
2  AFTER INSERT OR UPDATE OF P_ONHAND ON PRODUCT
3  BEGIN
4      UPDATE PRODUCT
5          SET P_REORDER = 1
6          WHERE P_ONHAND <= P_MIN;
7  END;
8  /
```

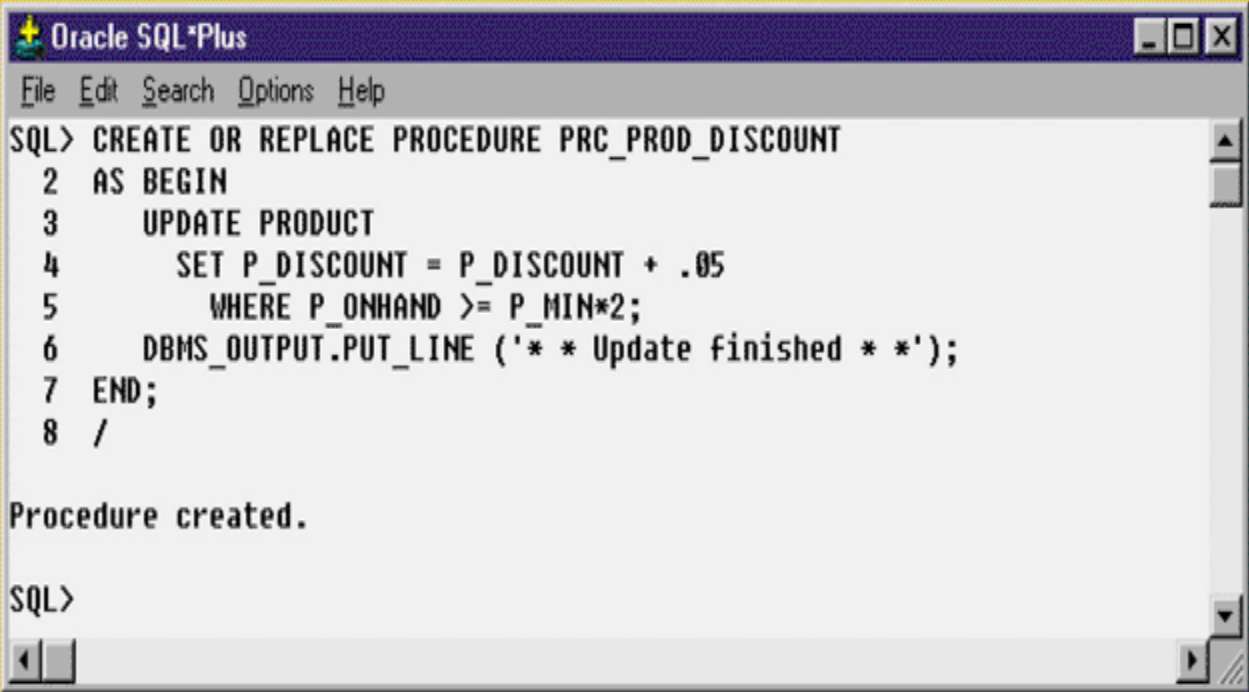
Below the commands, the message "Trigger created." is displayed. The prompt "SQL>" is visible at the bottom of the text area. The window has standard Windows-style window controls (minimize, maximize, close) in the top right corner.

Stored Procedures: Advantages

- ❑ Substantially reduce network traffic and increase performance
- ❑ No transmission of individual SQL statements over network
- ❑ Help reduce code duplication by means of code isolation and code sharing
- ❑ Minimize chance of errors and cost of application development and maintenance

Creating the PRC_PROD_DISCOUNT Stored Procedure

FIGURE 7.41 CREATING THE PRC_PROD_DISCOUNT STORED PROCEDURE



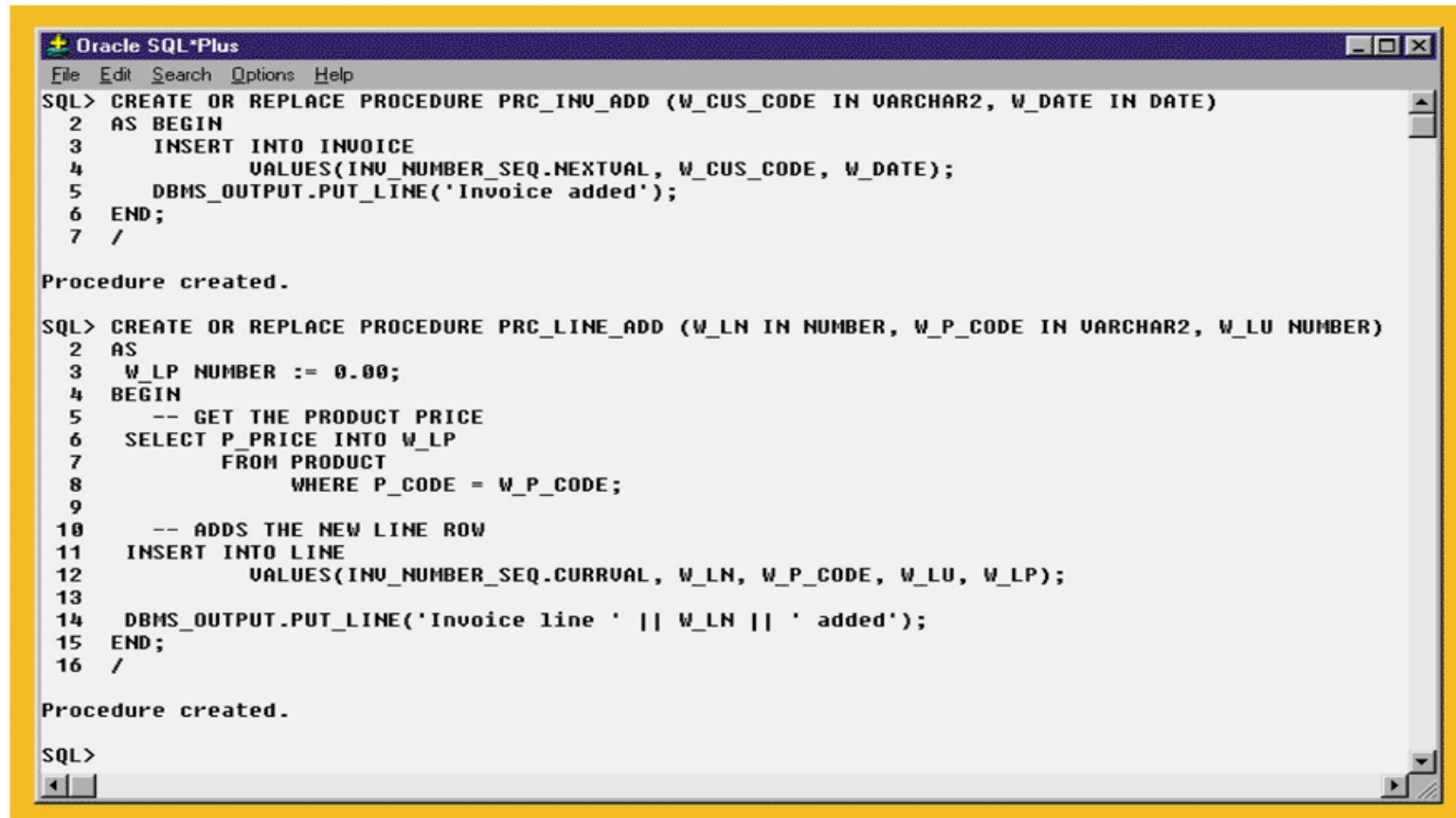
```
Oracle SQL*Plus
File Edit Search Options Help
SQL> CREATE OR REPLACE PROCEDURE PRC_PROD_DISCOUNT
2 AS BEGIN
3   UPDATE PRODUCT
4     SET P_DISCOUNT = P_DISCOUNT + .05
5     WHERE P_ONHAND >= P_MIN*2;
6   DBMS_OUTPUT.PUT_LINE ('* * Update finished * *');
7 END;
8 /

Procedure created.

SQL>
```

The PRC_INV_ADD and PRC_LINE_ADD Stored Procedures

FIGURE 7.46 THE PRC_INV_ADD AND PRC_LINE_ADD STORED PROCEDURES



```
Oracle SQL*Plus
File Edit Search Options Help
SQL> CREATE OR REPLACE PROCEDURE PRC_INV_ADD (W_CUS_CODE IN VARCHAR2, W_DATE IN DATE)
2 AS BEGIN
3     INSERT INTO INVOICE
4         VALUES(INV_NUMBER_SEQ.NEXTVAL, W_CUS_CODE, W_DATE);
5     DBMS_OUTPUT.PUT_LINE('Invoice added');
6 END;
7 /

Procedure created.

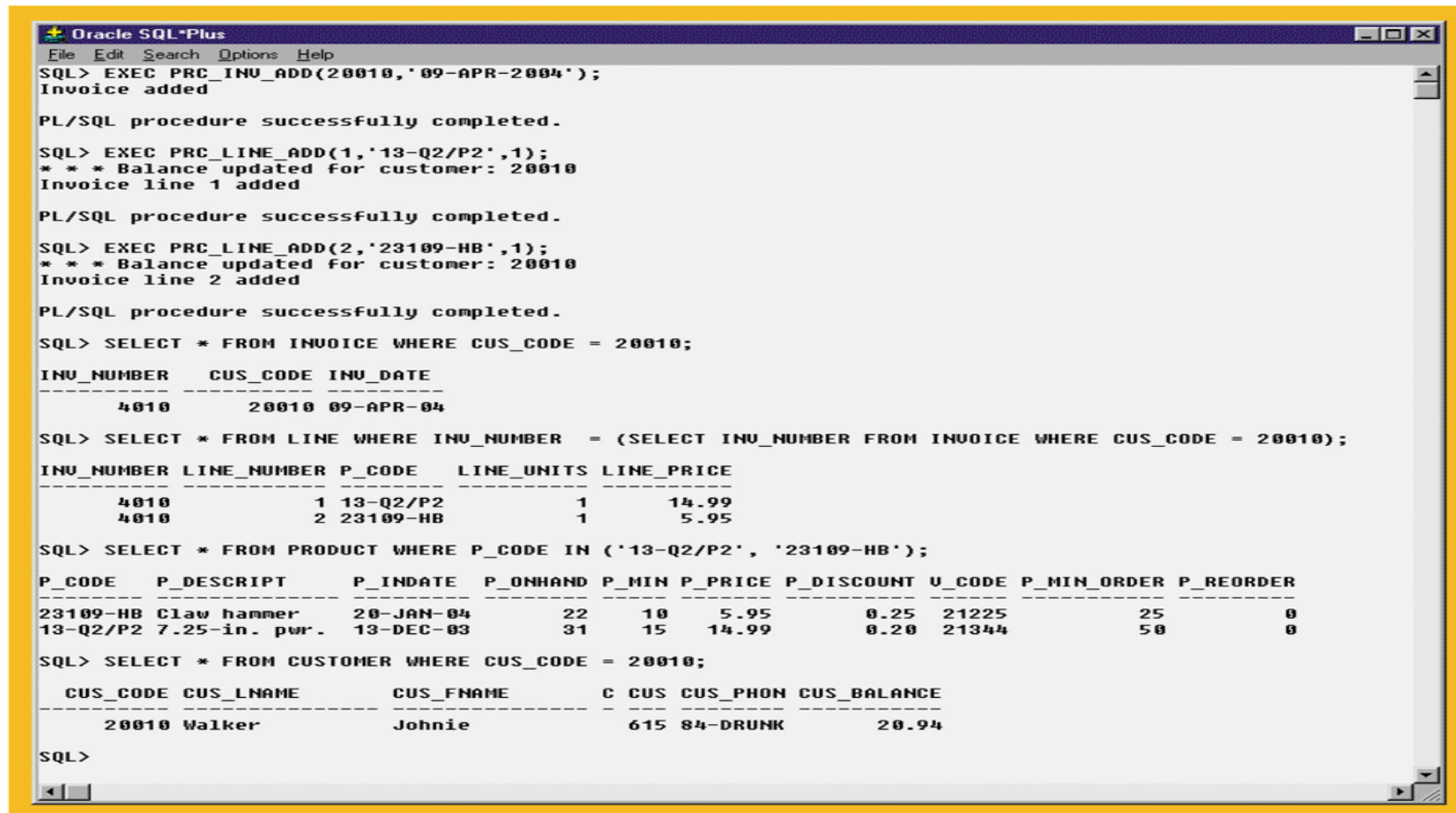
SQL> CREATE OR REPLACE PROCEDURE PRC_LINE_ADD (W_LN IN NUMBER, W_P_CODE IN VARCHAR2, W_LU NUMBER)
2 AS
3     W_LP NUMBER := 0.00;
4 BEGIN
5     -- GET THE PRODUCT PRICE
6     SELECT P_PRICE INTO W_LP
7         FROM PRODUCT
8         WHERE P_CODE = W_P_CODE;
9
10    -- ADDS THE NEW LINE ROW
11    INSERT INTO LINE
12        VALUES(INV_NUMBER_SEQ.CURRVAL, W_LN, W_P_CODE, W_LU, W_LP);
13
14    DBMS_OUTPUT.PUT_LINE('Invoice line ' || W_LN || ' added');
15 END;
16 /

Procedure created.

SQL>
```

Testing the PRC_INV_ADD and PRC_LINE_ADD Procedures

FIGURE 7.47 TESTING THE PRC_INV_ADD AND PRC_LINE_ADD PROCEDURES



```
Oracle SQL*Plus
File Edit Search Options Help
SQL> EXEC PRC_INV_ADD(20010,'09-APR-2004');
Invoice added

PL/SQL procedure successfully completed.

SQL> EXEC PRC_LINE_ADD(1,'13-Q2/P2',1);
* * * Balance updated for customer: 20010
Invoice line 1 added

PL/SQL procedure successfully completed.

SQL> EXEC PRC_LINE_ADD(2,'23109-HB',1);
* * * Balance updated for customer: 20010
Invoice line 2 added

PL/SQL procedure successfully completed.

SQL> SELECT * FROM INVOICE WHERE CUS_CODE = 20010;

 INU_NUMBER   CUS_CODE  INU_DATE
-----
      4010         20010  09-APR-04

SQL> SELECT * FROM LINE WHERE INU_NUMBER = (SELECT INU_NUMBER FROM INVOICE WHERE CUS_CODE = 20010);

 INU_NUMBER LINE_NUMBER P_CODE   LINE_UNITS LINE_PRICE
-----
      4010           1 13-Q2/P2           1       14.99
      4010           2 23109-HB           1        5.95

SQL> SELECT * FROM PRODUCT WHERE P_CODE IN ('13-Q2/P2', '23109-HB');

 P_CODE   P_DESCRIPTOR   P_INDATE   P_ONHAND P_MIN P_PRICE P_DISCOUNT U_CODE P_MIN_ORDER P_REORDER
-----
23109-HB Claw hammer      20-JAN-04        22     10    5.95         0.25  21225         25         0
13-Q2/P2 7.25-in. pwr.      13-DEC-03        31     15   14.99         0.20  21344         50         0

SQL> SELECT * FROM CUSTOMER WHERE CUS_CODE = 20010;

 CUS_CODE CUS_LNAME   CUS_FNAME   C CUS CUS_PHON CUS_BALANCE
-----
      20010 Walker      Johnie      615 84-DRUNK      20.94

SQL>
```


Embedded SQL

- ❑ SQL statements that are contained within an application programming language
- ❑ In some programming languages the SQL code is preceded by EXEC SQL and followed with END-EXEC
 - SQLSTATE and SQLCODE reporting variables inform the program of the status of the SQL code that executed

Dynamic SQL

- ❑ SQL statement is not known in advance, but instead is generated at run time
- ❑ Program can generate SQL statements at run time that are required to respond to ad hoc queries
- ❑ Attribute list and the condition are not known until the end user specifies them
- ❑ Tends to be much slower than static SQL
- ❑ Requires more computer resources